

CS301 — Algorithm Design Overview

CS301 — Algorithms & Data Structures

Slide 1: Slide 1: Course Overview — CS301 Algorithms

This course covers design and analysis of algorithms. Grading: 40% exams, 30% assignments, 30% project.

Slide 2: Slide 2: Big-O Notation

$O(f(n))$: upper bound. $\Omega(f(n))$: lower bound. $\Theta(f(n))$: tight bound.

$f(n) = O(g(n))$ iff $\exists c > 0, n_0$ s.t. $f(n) \leq c \cdot g(n) \forall n \geq n_0$

Common complexities (fastest to slowest):

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

Slide 3: Slide 3: Recurrences — The Master Theorem

$$T(n) = aT(n/b) + f(n)$$

$$\text{Case 1: } f(n) = O(n^{\log_b(a) - \epsilon}) \rightarrow T(n) = \Theta(n^{\log_b(a)})$$

$$\text{Case 2: } f(n) = \Theta(n^{\log_b(a)}) \rightarrow T(n) = \Theta(n^{\log_b(a)} \cdot \log n)$$

$$\text{Case 3: } f(n) = \Omega(n^{\log_b(a) + \epsilon}) \rightarrow T(n) = \Theta(f(n))$$

$$\text{Example: Merge Sort } T(n) = 2T(n/2) + n$$

$$a=2, b=2, \log_b(a) = 1, f(n) = n = \Theta(n^1) \rightarrow \text{Case 2} \rightarrow T(n) = \Theta(n \log n)$$

Slide 4: Slide 4: Divide and Conquer

Pattern:

1. Divide: split problem into subproblems
2. Conquer: solve subproblems recursively
3. Combine: merge solutions

Examples:

- Merge Sort: divide at midpoint, merge results
- Binary Search: divide at midpoint, recurse on one half
- Strassen's Matrix Multiplication: divide 2x2 blocks, 7 recursive calls $\rightarrow O(n^{2.81})$

Slide 5: Slide 5: Graph Algorithms — BFS

Breadth-First Search:

- Explores level by level (uses Queue)
- Finds shortest path in unweighted graphs
- Time: $O(V + E)$
- Space: $O(V)$

Applications: shortest path (unweighted), social network connections, web crawling, level-order tree traversal

Slide 6: Slide 6: Graph Algorithms — DFS

Depth-First Search:

- Explores as deep as possible (uses Stack or recursion)
- Time: $O(V + E)$, Space: $O(V)$

Applications:

- Topological sort (directed acyclic graphs)
- Cycle detection
- Finding strongly connected components (Kosaraju, Tarjan)
- Maze solving

Slide 7: Slide 7: Dijkstra's Algorithm

Shortest path in weighted graphs with NON-NEGATIVE edges.

Algorithm:

1. $\text{dist}[\text{source}] = 0$; $\text{dist}[v] = \infty$ for all other v
2. Priority queue: insert ($\text{dist}=0$, source)
3. While PQ not empty:
 - a. Extract vertex u with minimum $\text{dist}[u]$
 - b. For each neighbor v of u :
if $\text{dist}[u] + w(u,v) < \text{dist}[v]$: update $\text{dist}[v]$, push to PQ

Complexity:

- With binary heap: $O((V + E) \log V)$
- With Fibonacci heap: $O(E + V \log V)$

Does NOT work with negative-weight edges (use Bellman-Ford instead)

Slide 8: Slide 8: Bellman-Ford Algorithm

Shortest path allowing NEGATIVE edges (no negative cycles).

Algorithm:

1. $\text{dist}[\text{source}] = 0$; $\text{dist}[v] = \infty$ for all other v

2. Repeat $V-1$ times:

For each edge (u,v,w) : if $\text{dist}[u] + w < \text{dist}[v]$: relax

3. Check for negative cycles: do one more pass, if any relaxation succeeds $\rightarrow$ negative cycle exists

Complexity: $O(VE)$ — much slower than Dijkstra

Used when: graph has negative edges, need to detect negative cycles

Slide 9: Slide 9: Dynamic Programming Principles

DP: solve subproblems once and store results.

Requirements:

1. Optimal substructure: optimal solution contains optimal solutions to subproblems
2. Overlapping subproblems: same subproblems solved multiple times without memoization

Approaches:

- Top-down (memoization): recursive + cache
- Bottom-up (tabulation): fill table iteratively

Examples: Fibonacci, Knapsack, LCS, Edit Distance, Matrix Chain Multiplication

Slide 10: Slide 10: Classic DP — Longest Common Subsequence

$\text{LCS}(s1, s2)$: length of longest subsequence common to both strings.

Recurrence:

$\text{dp}[i][j] = \text{dp}[i-1][j-1] + 1$ if $s1[i] == s2[j]$

$\text{dp}[i][j] = \max(\text{dp}[i-1][j], \text{dp}[i][j-1])$ otherwise

Time: $O(mn)$, Space: $O(mn)$ or $O(\min(m,n))$ with optimization

Backtrace for actual subsequence: follow arrows in dp table from $\text{dp}[m][n]$ to $\text{dp}[0][0]$.

Slide 11: Slide 11: 0-1 Knapsack Problem

Given items with weights $w[i]$ and values $v[i]$, and knapsack capacity W :

Maximize total value with total weight $\leq W$, each item used at most once.

$dp[i][j]$ = max value using first i items with capacity j

$dp[i][j] = dp[i-1][j]$ if $w[i] > j$ (can't include item i)

$dp[i][j] = \max(dp[i-1][j], v[i] + dp[i-1][j - w[i]])$ otherwise

Time: $O(nW)$, Space: $O(nW)$ — pseudo-polynomial (W can be large)

This is NP-hard in general (exponential in the number of bits of W).

Slide 12: Greedy Algorithms

Greedy: make locally optimal choice at each step.

When does greedy work? Need greedy choice property + optimal substructure.

Examples:

1. Activity Selection: sort by finish time, always pick earliest finishing non-conflicting activity. Optimal.
2. Fractional Knapsack: sort by value/weight ratio, take greedily. Optimal.
3. Huffman Coding: build optimal prefix-free code using min-heap. Optimal.
4. 0-1 Knapsack: greedy FAILS (need DP).

Proving greedy: exchange argument (show any optimal solution can be transformed to greedy solution without decreasing value).